

ION-BPP — System Flow Diagrams

Beckn Protocol v2.0 | Business Provider Platform

Food & Beverage Vertical — Sarpino's Pizzeria

Generated: 2026-04-20

1. System Architecture Overview

Beckn Network BAP Buyer App Platform select / init / confirm on_* callbacks ION-BPP Express.js :4001 catalog/publish External Fabric Beckn Fabric fabric.nfh.global ION-BPP Application Internals Express.js Middleware (CORS / Log) select.js init.js confirm.js publish.js /resources/add /provider/add /publish /catalog/publish GET /providers /products /orders MongoDB — ION-bpp resources products / catalog items providers store profiles orders confirmed orders catalog_publishes publish audit log

2. Beckn Transaction Flow (select → init → confirm → status → track)

BAP Buyer App Platform ION-BPP :4001 MongoDB ION-bpp SELECT 1. POST /select 2. findOne(resourceId, offerId) 3. resource document calculate pricing: GST 5% + ₹25 packing + ₹49 delivery 4. ACK 5. on_select callback [async ~2s] INIT 6. POST /init (with delivery address) add payment methods: WALLET | COD | UPI · SLA: PT30M-PT2H 7. ACK 8. on_init callback [async] (+ payment methods, commitments: DRAFT) CONFIRM 9. POST /confirm (with payment method) generate orderId: ORD-FNB-{timestamp}-{seq} · set commitments ACTIVE 10. orders.insertOne() (status: CONFIRMED) 11. { orderId } 12. ACK 13. on_confirm callback [async] (orderId, status: ORDER_PLACED) STATUS 14. POST /status 15. ACK 16. on_status: PENDING [t + 2s] 17. updateOne({ status: EN_ROUTE }) 18. on_status: IN_ROUTE [t + 7s] 19. updateOne({ status: DELIVERED }) 20. on_status: DELIVERED [t + 12s] TRACK 21. POST /track 22. ACK 23. on_track callback [async] Legend Synchronous message Asynchronous / return message BAP ION-BPP MongoDB Note: All Beckn endpoints (/select /init /confirm /status /track) return ACK immediately. Actual responses fire asynchronously to http://localhost:8082/bpp/caller/{action}

3. Catalog Publishing Flow (register → publish → fabric)

Admin Operator ION-BPP :4001 MongoDB ION-bpp Beckn Fabric fabric.nfh.global STEP 1 — REGISTER DATA 1. POST /resources/add (resourceId, name, price, attributes...) 2. Resource.findOneAndUpdate() (upsert: true) 3. 201 { message: "ACK", resourceId: "..." } 4. POST /provider/add (providerId, name, charges, policies...) 5. Provider.findOneAndUpdate() (upsert: true) 6. 201 { message: "ACK", providerId: "..." } STEP 2 — TRIGGER PUBLISH 7. POST /publish { resourceIds: ["id1", "id2", ...] } 8. ACK (async processing begins) STEP 3 — BUILD

CATALOG (async) 9. Resource.find({ resourceId: { \$in: resourceIds } }) 10. resource documents []
11. Provider.findOne({ providerId }) 12. provider document { } assemble catalog/publish payload
(context + resources + offers) STEP 4 — DISPATCH TO FABRIC 13. POST /catalog/publish →
fabric.nfh.global 14. ACK (fabric response) 15. CatalogPublish.create() (audit log) 16.
Resource.updateMany({ isPublished: true }) 17. callback fired to callbackUrl (publish complete)
Note: Steps 9–17 execute asynchronously after the ACK in step 8. The POST /publish endpoint
returns immediately.